

Competitive Programming Notebook

py.Sandu

Contents

1	Graph	2
1.1	Dial	2
1.2	Bellmanford	2
1.3	Kruskal	2
1.4	Floydwarshall	2
1.5	Eulereanpath	3
1.6	Dijkstra	3
1.7	Kahn	3
1.8	Lca	3
2	Geometry	4
2.1	Convexhull	4
2.2	Point	4
2.3	Pointinpolygon	5
3	.template	5
3.1	Template	5
4	String	5
4.1	Hash	5
5	DP	6
5.1	Lis	6
5.2	Countingdp	6
5.3	Knapsack	6
6	Math	6
6.1	Extendedeuclidean	6
6.2	Crivo	6
6.3	Fastexponentiation	7
6.4	Matrixexponentiation	7
6.5	Matrixmultiplication	7
6.6	Pie	7
6.7	Factorial	7
7	DS	7
7.1	Bit2d	7
7.2	Segtree	8
7.3	Seglazy	8
7.4	Psum2d	8
7.5	Dsu	9
7.6	Sparsetable	9
7.7	Bit	9
7.8	Monotonicstack	10

1 Graph

1.1 Dial

```

1 // Dial, optimized dijkstra for low W
2 // Time-complexity: O(E + V*W), W : max weight
3
4 vector<int> dial(int n, int w, int src, vector<vector<pii>> &adj){
5
6     vector<int> dist(n+1, INF);
7     vector<queue<int>> a(w+1);
8     dist[src] = 0; a[0].push(src);
9     int pos = 0, qnt = 1;
10
11     while(qnt){
12         while(a[pos%(w+1)].empty()) pos++;
13
14         int i = a[pos%(w+1)].front(); a[pos%(w+1)].pop(); qnt--;
15
16         if(dist[i] < pos) continue;
17
18         for(auto [u, d_u]: adj[i]){
19             if(dist[i] + d_u < dist[u]){
20                 dist[u] = dist[i] + d_u;
21                 a[(dist[u])%(w+1)].push(u); qnt++;
22             }
23         }
24     }
25
26     return dist;
27 }
28
29 // especial case: max W = 1
30 vector<int> bfs01(int n, int src, vector<vector<pii>> &adj){
31
32     vector<int> dist(n+1, INF);
33     deque<int> dq;
34
35     dist[src] = 0;
36     dq.push_front(src);
37
38     while(!dq.empty()){
39         int i = dq.front();
40         dq.pop_front();
41
42         for(auto [u, w] : adj[i]){
43             if(dist[i] + w < dist[u]){
44                 dist[u] = dist[i] + w;
45                 if(w == 0) dq.push_front(u);
46                 else dq.push_back(u);
47             }
48         }
49     }
50
51     return dist;
52 }
```

1.2 Bellmanford

```

1 // Find the shortest path from a source vertex to all
   other vertices in a graph with negative weight
   edges and detect negative weight cycles.
2 // Time Complexity: O(V * E)
3 // Space Complexity: O(V)
4 // Use cases: SSSP with negative edges, negative
   cycle detection, system of difference constraints
   , shortest path with at most K edges
5
```

```

6 bool bellmanFord(vector<vector<pair<int, int>>> &adj,
   int src) {
7
8     int n = adj.size() - 1;
9     vector<int> dist(n+1, INF);
10    dist[src] = 0;
11
12    // Relax all edges n-1 times
13    for (int i = 1; i < n; i++) {
14        for (int u = 1; u <= n; u++) {
15            for (auto& [v, w] : adj[u]) {
16                if (dist[u] != INF and dist[u] + w <
17                    dist[v]) {
18                    dist[v] = dist[u] + w;
19                }
20            }
21        }
22    }
23
24    // Check for negative weight cycles
25    for (int u = 1; u <= n; u++) {
26        for (auto& [v, w] : adj[u]) {
27            if (dist[u] != INF and dist[u] + w < dist
28                [v]) {
29                // Graph contains negative weight
30                cycle
31                return true;
32            }
33        }
34    }
35
36    return false;
37 }
```

1.3 Kruskal

```

1 // Kruskal algorithm for finding the Minimum Spanning
   Tree
2 // Time Complexity: O(E log E)
3 // Space complexity: O(V + E)
4 // Use cases: connect nodes with min/max cost,
   clustering, minimax path
5
6 // edges: {weight, u, v}
7 vector<vector<pair<int, int>>> kruskalMST(vector<
   array<int, 3>> &edges, int n){
8     int m = (int)edges.size();
9     sort(edges.begin(), edges.end());
10    vector<vector<pair<int, int>>> mst(n+1);
11    DSU dsu(n+1); int cost = 0;
12    for(int i=0; i<m; i++){
13        int w = edges[i][0], a = edges[i][1], b =
14        edges[i][2];
15        if(dsu.merge(a, b)){
16            mst[a].push_back({w, b});
17            mst[b].push_back({w, a});
18            cost += w;
19        }
20    }
21    return mst;
22 }
```

1.4 Floydwarshall

```

1 // Find the shortest paths in a weighted graph with
   positive or negative edge weights (but with no
   negative cycles).
2 // Time Complexity: O(V^3)
3 // Space Complexity: O(V^2)
4 // Use cases: reachability, minimax distance, global
   negative cycle detection(dist[i][i] < 0)
5
```

```

6 void floydWarshall(vector<vector<int>>& dist) { //
    dist(INF)
7
8     int n = dist.size() - 1;
9     for(int i = 1; i <= n; i++) dist[i][i] = 0;
10
11     for (int k = 1; k <= n; k++) {
12         for (int i = 1; i <= n; i++) {
13             for (int j = 1; j <= n; j++) {
14                 if (dist[i][k] != INF and dist[k][j]
15                     != INF) {
16                     dist[i][j] = min(dist[i][j], dist
17                         [i][k] + dist[k][j]);
18                 }
19             }
20 }

```

1.5 Eulereanpath

```

1 // Find the Eulerian path in a graph using Hierholzer
  // 's algorithm
2 // Time complexity: O(E) where E is the number of
  // edges in the graph
3 // Use cases: visit every edge once, DNA sequencing,
  // puzzles, drawing figures
4
5 vector<int> findEulerianPath(vector<vector<int>>& adj
  , int start) {
6
7     vector<int> path;
8     stack<int> s;
9     s.push(start);
10
11     while (!s.empty()) {
12         int i = s.top();
13         if (adj[i].empty()) {
14             path.push_back(i);
15             s.pop();
16         } else {
17             int u = adj[i].back(); adj[i].pop_back();
18             s.push(u);
19         }
20     }
21
22     reverse(path.begin(), path.end());
23     return path;
24 }

```

1.6 Dijkstra

```

1 // Find shortest path from a source vertex to all
  // other vertices in a graph with non-negative edge
  // weights
2 // Time Complexity: O((V + E) log V)
3 // Space Complexity: O(V)
4 // Use cases: shortest path in non-negative weighted
  // graphs
5
6 vector<int> dijkstra(vector<vector<pair<int, int>>>&
  adj, int src) {
7
8     int n = adj.size() - 1;
9     vector<int> dist(n+1, INF);
10    dist[src] = 0;
11
12    priority_queue<pair<int, int>, vector<pair<int,
  int>>, greater<pair<int, int>>> pq; // Min-heap
  priority queue
13    pq.push({0, src});
14

```

```

15    while (!pq.empty()) {
16
17        auto [d, i] = pq.top();
18        pq.pop();
19
20        if(d > dist[i]) continue;
21
22        for (auto [u, w] : adj[i]) {
23
24            if (dist[i] + w < dist[u]) {
25                dist[u] = dist[i] + w;
26                pq.push({dist[u], u});
27            }
28        }
29    }
30
31    return dist;
32 }

```

1.7 Kahn

```

1 // Topological Sort (Kahn's Algorithm)
2 // Time Complexity: O(V + E)
3 // Space Complexity: O(V)
4 // Use cases: Dependency resolution, cycle detection
  // in directed graphs, DP on DAGs
5
6 vector<int> TopologicalSort(vector<vector<int>>& adj,
  int n){
7     vector<int> degree(n+1, 0);
8     for(int i=1; i<=n; i++){
9         for(auto u: adj[i]) degree[u]++;
10    }
11
12    // priority_queue in case of lexicographically
  // smallest answer
13    queue<int> q; vector<int> ans;
14    for(int i=1; i<=n; i++){
15        if(degree[i] == 0){
16            q.push(i); ans.push_back(i);
17        }
18    }
19
20
21    while(!q.empty()){
22        int i = q.front(); q.pop();
23        for(auto u: adj[i]){
24            degree[u]--;
25            if(degree[u] == 0){
26                q.push(u); ans.push_back(u);
27            }
28        }
29    }
30
31    if((int)ans.size() != n) return {};
32    return ans;
33
34 }

```

1.8 Lca

```

1
2 // Pre-processing: O(N log N) | Query: O(log N)
3
4 int timer, k;
5 vector<vector<int>> adj;
6 vector<vector<int>> up;
7 vector<int> tin, tout;
8
9 void dfs(int v, int p){
10     tin[v] = ++timer;
11     up[v][0] = p;

```

```

12     for(int i=1; i <= k; i++){
13         up[v][i] = up[up[v][i-1]][i-1];
14     }
15     for(int u: adj[v]){
16         if(u != p) dfs(u, v);
17     }
18     tout[v] = ++timer;
19 }
20
21 bool is_ancestor(int u, int v){
22     return tin[u] <= tin[v] and tout[u] >= tout[v];
23 }
24
25 int lca(int u, int v){
26     if(is_ancestor(u, v)) return u;
27     if(is_ancestor(v, u)) return v;
28
29     for(int i=k; i >= 0; i--){
30         if(!is_ancestor(up[u][i], v)) u = up[u][i];
31     }
32     return up[u][0];
33 }
34
35 void pre_process(int n, int root){
36     tin.resize(n+1);
37     tout.resize(n+1);
38     timer = 0;
39     k = __lg(n+1);
40     up.assign(n+1, vector<int>(k+1));
41     dfs(root, root);
42 }

```

2 Geometry

2.1 Convexhull

```

1
2 // pontos colineares na borda, mude <= EPS para < -
   EPS.
3 vector<Point> convex_hull(vector<Point>& pts) {
4     int n = pts.size(), k = 0;
5     if (n <= 2) return pts;
6
7     vector<Point> h(2 * n);
8     sort(pts.begin(), pts.end());
9
10    // lower hull
11    for (int i = 0; i < n; ++i) {
12        while (k >= 2 && cross_product(h[k-2], h[k-1], pts[i]) <= EPS) k--;
13        h[k++] = pts[i];
14    }
15
16    // upper hull
17    for (int i = n - 2, t = k + 1; i >= 0; i--) {
18        while (k >= t && cross_product(h[k-2], h[k-1], pts[i]) <= EPS) k--;
19        h[k++] = pts[i];
20    }
21
22    h.resize(k - 1); // remove o ponto duplicado no final
23    return h;
24 }

```

2.2 Point

```

1
2 const double EPS = 1e-9;
3 // EPS = 0 -> int
4

```

```

5 struct Point {
6     double x, y;
7
8     Point() : x(0), y(0) {}
9     Point(double x, double y) : x(x), y(y) {}
10
11     Point operator+(const Point& p) const { return
12         Point(x + p.x, y + p.y); }
13     Point operator-(const Point& p) const { return
14         Point(x - p.x, y - p.y); }
15     Point operator*(double c) const { return Point(x
16         * c, y * c); }
17     Point operator/(double c) const { return Point(x
18         / c, y / c); }
19
20     bool operator<(const Point& p) const {
21         if (abs(x - p.x) > EPS) return x < p.x;
22         return y < p.y - EPS;
23     }
24 };
25
26 // > 0: B esq. vetor OA, < 0: B dir. vetor OA, 0:
   Colineares
27 double cross_product(Point O, Point A, Point B) {
28     return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) *
29         (B.x - O.x);
30 }
31
32 // distancia euclidiana
33 double dist(Point A, Point B) {
34     return hypot(A.x - B.x, A.y - B.y);
35 }
36
37 // distancia quadrada
38 double dist2(Point A, Point B) {
39     return (A.x - B.x) * (A.x - B.x) + (A.y - B.y) *
40         (A.y - B.y);
41 }
42
43 // produto escalar, dot == 0 perpendiculares
44 double dot_product(Point O, Point A, Point B) {
45     return (A.x - O.x) * (B.x - O.x) + (A.y - O.y) *
46         (B.y - O.y);
47 }
48
49 int get_half(Point O, Point P) {
50     if (P.y - O.y > EPS or (abs(P.y - O.y) <= EPS and
51         P.x - O.x > EPS)) return 1;
52     return 0;
53 }
54
55 // ordenacao por perimetro
56 void sort_by_angle(vector<Point>& p, Point center) {
57     sort(p.begin(), p.end(), [&](const Point& A,
58         const Point& B) {
59         int halfA = get_half(center, A);
60         int halfB = get_half(center, B);
61
62         if (halfA != halfB) {
63             return halfA > halfB;
64         }
65
66         double cross = cross_product(center, A, B);
67         if (abs(cross) > EPS) {
68             return cross > 0;
69         }
70
71         return dist2(center, A) < dist2(center, B) -

```

```

67     EPS;
68     });
69 }
70 // shoelace formula, vÃrtices do polÃgono ordenados
71 // pelo perÃmetro.
72 double polygon_area(const vector<Point>& p) {
73     double area = 0.0;
74     int n = p.size();
75     for (int i = 0; i < n; i++) {
76         int next_i = (i + 1) % n;
77         area += (p[i].x * p[next_i].y) - (p[next_i].x
78             * p[i].y);
79     }
80     // return abs(area) -> int
81     return abs(area) / 2.0;
82 }
83
84 // aux func
85 int orientation(Point O, Point A, Point B) {
86     double val = cross_product(O, A, B);
87     if (abs(val) <= EPS) return 0; // Colinear
88     return (val > 0) ? 1 : -1; // 1: Esquerda,
89     -1: Direita
90 }
91 // 2. P in seg AB?
92 bool on_segment(Point P, Point A, Point B) {
93     return P.x >= min(A.x, B.x) - EPS && P.x <= max(A
94         .x, B.x) + EPS &&
95         P.y >= min(A.y, B.y) - EPS && P.y <= max(A
96         .y, B.y) + EPS;
97 }
98 // Ab seg cross CD?
99 bool segments_intersect(Point A, Point B, Point C,
100     Point D) {
101     // Calcula as 4 orientaÃÃes necessÃrias
102     int o1 = orientation(A, B, C);
103     int o2 = orientation(A, B, D);
104     int o3 = orientation(C, D, A);
105     int o4 = orientation(C, D, B);
106
107     if (o1 != o2 && o3 != o4) {
108         return true;
109     }
110
111     if (o1 == 0 && on_segment(C, A, B)) return true;
112     // C toca AB
113     if (o2 == 0 && on_segment(D, A, B)) return true;
114     // D toca AB
115     if (o3 == 0 && on_segment(A, C, D)) return true;
116     // A toca CD
117     if (o4 == 0 && on_segment(B, C, D)) return true;
118     // B toca CD
119
120     return false; // NÃo se cruzam
121 }

```

2.3 Pointinpolygon

```

1 // retorna: 1 (Dentro), -1 (Na borda), 0 (Fora)
2 // o polÃgono nÃo precisa ser convexo, mas os
3 // vÃrtices devem estar em ordem.
4 int point_in_polygon(Point P, const vector<Point>&
5     poly) {
6     int n = poly.size();
7     bool in = false;
8
9     for (int i = 0; i < n; i++) {
10         Point A = poly[i];

```

```

9         Point B = poly[(i + 1) % n];
10         if (on_segment(P, A, B)) return -1;
11         if (A.y > B.y) swap(A, B);
12         if (A.y <= P.y + EPS && P.y + EPS < B.y) {
13             if (cross_product(A, B, P) > EPS) {
14                 in = !in;
15             }
16         }
17     }
18     return in ? 1 : 0;
19 }
20 }

```

3 .template

3.1 Template

```

1 #include<bits/stdc++.h>
2
3 using namespace std;
4
5 #define int long long
6 #define vi vector<int>
7 #define pii pair<int, int>
8 #define all(x) (x).begin(), (x).end()
9 #define debug(x...) cout<<"[#x]: ", [(auto...$)]{((
10     cout<<$<<" "; ...});}(x),cout<<endl
11
12 void solve(){
13 }
14
15 signed main(){
16     ios::sync_with_stdio(false);
17     cin.tie(NULL);
18     int t = 1;
19     //cin >> t;
20     while(t--) solve();
21     return 0;
22 }

```

4 String

4.1 Hash

```

1 // String Hash template
2 // constructor(s) - O(|s|)
3 // query(l, r) - returns the hash of the range [l,r]
4 // from left to right - O(1)
5 // patrocinado por tiagodfs
6
7 mt19937 rng(time(nullptr));
8
9 struct Hash {
10     const int X = rng();
11     const int MOD = 1e9+7;
12     int n; string s;
13     vector<int> h, hi, p;
14     Hash() {}
15     Hash(string s): s(s), n(s.size()), h(n), hi(n), p
16         (n) {
17         for (int i=0;i<n;i++) p[i] = (i ? X*p[i-1]:1)
18             % MOD;
19         for (int i=0;i<n;i++)
20             h[i] = (s[i] + (i ? h[i-1]:0) * X) % MOD;
21         for (int i=n-1;i>=0;i--)
22             hi[i] = (s[i] + (i+1<n ? hi[i+1]:0) * X)
23                 % MOD;
24     }
25     int query(int l, int r) {

```

```

23     int hash = (h[r] - (1 ? h[l-1]*p[r-l+1]%MOD : 18
0));
24     return hash < 0 ? hash + MOD : hash;
25 }
26 int query_inv(int l, int r) {
27     int hash = (hi[l] - (r+1 < n ? hi[r+1]*p[r-l
+1] % MOD : 0));
28     return hash < 0 ? hash + MOD : hash;
29 }
30 };

```

5 DP

5.1 Lis

```

1 // Find the length of the longest increasing
  subsequence in a given array
2 // Time complexity: O(n log n) where n is the length
  of the input array
3
4 int lis(vector<int>& arr) {
5     vector<int> dp;
6     for (int x : arr) {
7         auto it = lower_bound(all(dp), x);
8         if(it == dp.end()) dp.push_back(x);
9         else *it = x;
10    }
11    return dp.size();
12 }

```

5.2 Countingdp

```

1 // Counting DP (Sum of all ways / Unbounded Knapsack)
2 // Time Complexity: O(N * Target)
3 // Space Complexity: O(Target)
4 // Use case: Count number of ways to form a sum
5
6 int count_ways(vector<int>& items, int target) {
7
8     vector<int> dp(target + 1, 0);
9     dp[0] = 1;
10
11    for (int item : items) {
12        for (int i = item; i <= target; i++) {
13            dp[i] = (dp[i] + dp[i - item]) % MOD;
14        }
15    }
16
17    return dp[target];
18 }

```

5.3 Knapsack

```

1 // General take/not take dp problem
2 // Time Complexity: aprox. O(n * m)
3 // Space Complexity: O(n * m) / O(m)
4 // Use cases: maximize/minimize sum
5
6 // Top-down
7 int knapsack_top_down(vector<int>& v, vector<int>& w,
  int x){
8     int n = (int)v.size();
9     vector<vector<int>>> dp(n, vector<int>(x+1, -1));
10    auto rec = [&](auto self, int i, int m) -> int {
11        if(i >= n || m == 0) return 0;
12        if(dp[i][m] != -1) return dp[i][m];
13        int take = 0, not_take = self(self, i+1, m);
14        if(m >= w[i]){
15            take = v[i] + self(self, i+1, m - w[i]);
16        }
17        return dp[i][m] = max(take, not_take);

```

```

    };
19
20     return rec(rec, 0 , x);
21 }
22
23 // Bottom-up
24 int knapsack_bottom_up(vector<int>& v, vector<int>& w
  , int x){
25     int n = (int)v.size();
26     vector<int> dp(x+1, 0);
27
28     for(int i=0; i < n; i++){
29         for(int m=x; m >= w[i]; m--){
30             dp[m] = max(dp[m], v[i] + dp[m - w[i]]);
31         }
32     }
33
34     return dp[x];
35 }

```

6 Math

6.1 Extendedeuclidean

```

1 // Extended Euclidean Algorithm
2 // Returns (gcd(a, b), x, y) such that a*x + b*y =
  gcd(a, b)
3
4 int euclides(int a, int b, int &x, int &y){
5     if(b == 0){
6         x = 1, y = 0;
7         return a;
8     }
9     int x1, y1;
10    int g = euclides(b, a%b, x1, y1);
11    x = y1; y = x1 - y1*(a/b);
12    return g;
13 }
14
15 int inverso_mod(int a, int m){
16     int x, y;
17     int g = euclides(a, m, x, y);
18     if(g != 1) return -1;
19     return (x%m + m) % m;
20 }

```

6.2 Crivo

```

1
2 // Time complexity: O(n log log n)
3 vector<int> spf(limit+1, 0);
4 void findPrimes(){
5     iota(all(spf), 0);
6     for(int i = 2; i*i <= limit; i++){
7         if(spf[i] == i){
8             for(int j=i*i; j <= limit; j += i){
9                 if(spf[j] == j) spf[j] = i;
10            }
11        }
12    }
13    // prime: spf[i] == i
14 }
15
16 // Time complexity: O(log n)
17 vector<int> getFactors(int x){
18     vector<int> factors;
19     while(x > 1){
20         factors.push_back(spf[x]);
21         x /= spf[x];
22     }
23     return factors;

```

```

24 }
25
26 // Time complexity: O(n log n)
27 vi tot_div(limit+1, 0);
28 void count_mult(){
29     for(int i=1; i <= limit; i++){
30         for(int j=i; j <= limit; j += i){
31             tot_div[j]++;
32         }
33     }
34 }

```

6.3 Fastexponentiation

```

1 // Calculates (x^n) % MOD
2 // Time Complexity: O(log n)
3 // Space Complexity: O(1)
4
5 int fexp(int x, int b, int MOD){
6     int res = 1;
7
8     while(b > 0){
9         if(b & 1) res = (res * x) % MOD;
10        x = (x * x) % MOD;
11        b >>= 1;
12    }
13    return res;
14 }

```

6.4 Matrixexponentiation

```

1 // Calculate the Nth Fibonacci number using matrix
  exponentiation.
2 // Time Complexity: O(log N)
3 // Space Complexity: O(1)
4
5 void matrixExponentiation(vector<vector<long long>>&
  matrix, int power) {
6     int n = matrix.size();
7     vector<vector<long long>> result(n, vector<long
  long>(n, 0));
8
9     // Initialize result as the identity matrix
10    for (int i = 0; i < n; i++) {
11        result[i][i] = 1;
12    }
13
14    while (power) {
15        if (power % 2 == 1) {
16            matrixMultiplication(result, matrix,
  result);
17        }
18        matrixMultiplication(matrix, matrix, matrix);
19        power /= 2;
20    }
21
22    matrix = result;
23 }

```

6.5 Matrixmultiplication

```

1 // Multiply two matrices A and B, and return the
  result in matrix C.
2 // Time Complexity: O(n^3)
3 // Space Complexity: O(n^2)
4
5 vector<vector<int>> matrixMultiplication(const vector
  <vector<int>>& A, const vector<vector<int>>& B) {
6     int n = A.size();
7     vector<vector<int>> C(n, vector<int>(n, 0)); //
  Initialize result matrix with zeros
8

```

```

9     for (int i = 0; i < n; i++) {
10        for (int j = 0; j < n; j++) {
11            for (int k = 0; k < n; k++) {
12                C[i][j] += A[i][k] * B[k][j];
13            }
14        }
15    }
16    return C;
17 }

```

6.6 Pie

```

1 // inclusion&exclusion principle using bitmaks
2 int count_union(vi &tot, vi &conj){
3     int m = (int)conj.size(), ans = 0;
4     int mx = (1ll << m) - 1;
5     for(int i=mx; i > 0; i--){
6         int cnt = __builtin_popcount(i), prod = 1;
7         for(int j=0; j<m; j++){
8             if(1 & (i >> j)) prod *= conj[j];
9         }
10
11        if(prod >= (int)tot.size()) continue;
12        if(cnt%2 == 0) ans -= tot[prod];
13        else ans += tot[prod];
14    }
15    return ans;
16 }

```

6.7 Factorial

```

1 // Time Complexity: O(N) for precompute, O(1) for
  choose and perm
2
3 const int MAXN, MOD;
4 vector<int> fact(MAXN), ifact(MAXN);
5
6 void precompute(){
7     fact[0] = 1; ifact[0] = 1;
8     for(int i=1; i<MAXN; i++) fact[i] = (i * fact[i
  -1]) % MOD;
9     ifact[MAXN-1] = fexp(fact[MAXN-1], MOD-2, MOD);
10    for(int i=MAXN-2; i >= 0; i--) ifact[i] = (ifact[
  i+1]*(i+1)) % MOD;
11 }
12
13 int choose(int n, int r){
14     if(r < 0 or r > n) return 0;
15     int num = fact[n];
16     int den = (ifact[r] * ifact[n - r]) % MOD;
17     return (num * den) % MOD;
18 }
19
20 int perm(int n, int r){
21     if(r < 0 or r > n) return 0;
22     return (fact[n] * ifact[n - r]) % MOD;
23 }

```

7 DS

7.1 Bit2d

```

1 // Time complexity: query and update log N * log M
2 // 1-based indexing
3
4 struct BIT2D {
5
6     int n, m; vector<vector<int>> bit;
7     BIT2D(int n, int m) {
8         this->n = n;
9         this->m = m;

```

```

10     bit.assign(n+1, vector<int>(m+1, 0));
11 }
12
13 void update(int x, int y, int val) {
14     for (int i = x; i <= n; i += i & -i) {
15         for (int j = y; j <= m; j += j & -j) {
16             bit[i][j] += val;
17         }
18     }
19 }
20
21 int query(int x, int y) {
22     int sum = 0;
23     for (int i = x; i > 0; i -= i & -i) {
24         for (int j = y; j > 0; j -= j & -j) {
25             sum += bit[i][j];
26         }
27     }
28     return sum;
29 }
30
31 int queryRange(int x1, int y1, int x2, int y2) {
32     return query(x2, y2)
33         - query(x1 - 1, y2)
34         - query(x2, y1 - 1)
35         + query(x1 - 1, y1 - 1);
36 }
37
38 };

```

7.2 Segtree

```

1  template <typename T>
2
3  struct SegTree{
4      int n;
5      vector<T> tree;
6      const T NEUTRO = 1e18;
7
8      T merge(T a, T b) {return min(a, b);}
9      SegTree(int n){
10         this->n = n;
11         tree.assign(2*n, NEUTRO);
12     }
13
14     void build(vector<T>& v){
15         for(int i=0; i<n; i++) tree[n+i] = v[i];
16         for(int i=n-1; i>0; i--) tree[i] = merge(tree
17 [2*i], tree[2*i + 1]);
18     }
19
20     void update(int pos, T val){
21         pos += n;
22         tree[pos] = val;
23         for(pos /= 2; pos > 0; pos /= 2) tree[pos] =
24 merge(tree[pos*2], tree[pos*2 + 1]);
25     }
26
27     T query(int l, int r){
28         T res_l = NEUTRO, res_r = NEUTRO;
29         for(l += n, r += n; l <= r; l /= 2, r /= 2){
30             if(l%2 != 0) res_l = merge(res_l, tree[l
31 ++]);
32             if(r%2 == 0) res_r = merge(tree[r--],
33 res_r);
34         }
35         return merge(res_l, res_r);
36     }
37 };

```

7.3 Seglazy

```

1  template <typename T>
2
3  struct SegLazy{
4      int n;
5      vector<T> tree, lazy;
6      const T NEUTRO = 0, LAZY_NEUTRO = 0;
7
8      T merge(T a, T b) {return a + b;}
9
10     SegLazy(int n){
11         this->n = n;
12         tree.assign(4*n, NEUTRO);
13         lazy.assign(4*n, NEUTRO);
14     }
15
16     void push(int node, int l, int r){
17         if(lazy[node] == LAZY_NEUTRO) return;
18         tree[node] += lazy[node] * (r - l + 1);
19         if(l != r){
20             lazy[2*node] += lazy[node];
21             lazy[2*node + 1] += lazy[node];
22         }
23         lazy[node] = LAZY_NEUTRO;
24     }
25
26     void build(int node, int l, int r, vector<T> &v){
27         if(l == r){
28             tree[node] = v[l]; return;
29         }
30         int mid = (l+r)/2;
31         build(2*node, l, mid, v);
32         build(2*node + 1, mid+1, r, v);
33         tree[node] = merge(tree[2*node], tree[2*node
34 + 1]);
35     }
36
37     void update(int node, int l, int r, int ql, int
38 qr, T val){
39         push(node, l, r);
40         if(ql > r or qr < l) return;
41         if(ql <= l and r <= qr){
42             lazy[node] += val;
43             push(node, l, r);
44             return;
45         }
46         int mid = (l+r)/2;
47         update(2*node, l, mid, ql, qr, val);
48         update(2*node + 1, mid+1, r, ql, qr, val);
49         tree[node] = merge(tree[2*node], tree[2*node
50 + 1]);
51     }
52
53     T query(int node, int l, int r, int ql, int qr){
54         push(node, l, r);
55         if(ql > r or qr < l) return NEUTRO;
56         if(ql <= l and r <= qr) return tree[node];
57         int mid = (l+r)/2;
58         return merge(query(2*node, l, mid, ql, qr),
59 query(2*node + 1, mid + 1, r, ql, qr));
60     }
61
62     void build(const vector<T>& v) { build(1, 0, n -
63 1, v); }
64
65     void update(int ql, int qr, T val) { update(1, 0,
66 n - 1, ql, qr, val); }
67
68     T query(int ql, int qr) { return query(1, 0, n -
69 1, ql, qr); }
70
71 };

```

7.4 Psum2d


```

1 // Psum2D
2 // Time Complexity: Query  $O(1)$ 
3 // Space Complexity:  $O(N \cdot M)$ 
4 // Use cases: static 2d range sum, fixed-size window
   search, max sum submatrix
5
6 int n, m;
7 vector<vector<int>> arr(n+1, vector<int>(m+1, 0));
8 vector<vector<int>> psum(n+1, vector<int>(m+1, 0));
9
10 for (int i = 1; i <= n; i++) {
11     for (int j = 1; j <= m; j++) {
12         cin >> arr[i][j];
13         psum[i][j] = arr[i][j]
14             + psum[i][j-1]
15             + psum[i-1][j]
16             - psum[i-1][j-1];
17     }
18 }
19
20 // canto superior esquerdo (x1, y1) canto inferior
   direito (x2, y2)
21 int x1, y1, x2, y2;
22 int res = psum[x2][y2]
23     - psum[x1 - 1][y2]
24     - psum[x2][y1 - 1]
25     + psum[x1 - 1][y1 - 1];

```

7.5 Dsu

```

1 // Disjoint Set Union (DSU)
2 // Time Complexity: aprox.  $O(1)$ 
3 // Space Complexity:  $O(n)$ 
4 // Use cases: connected components, cycle detection,
   offline edge deletion
5
6 struct DSU{
7
8     vector<int> parent, size;
9
10     DSU(int n){
11         parent.assign(n+1, 0);
12         size.assign(n+1, 1);
13         iota(parent.begin(), parent.end(), 0);
14     }
15
16     int find(int v){
17         if(v == parent[v]) return v;
18         return parent[v] = find(parent[v]);
19     }
20
21     bool same(int a, int b){
22         return find(a) == find(b);
23     }
24
25     bool merge(int a, int b){
26         a = find(a);
27         b = find(b);
28         if(a != b){
29             if(size[a] < size[b]) swap(a, b);
30             parent[b] = a;
31             size[a] += size[b];
32             return true;
33         }
34         return false;
35     }
36 }
37 };

```

7.6 Sparsetable

```

1 // Sparse Table

```

```

2 // Time Complexity:  $O(n \cdot \log n)$  build,  $O(1)$  query
3 // Space complexity:  $O(n \cdot \log n)$ 
4 // Idempotent operation  $f(a, a) = a$  (min, max, gcd,
   and, or)
5 // Use cases: idempotent functions range queries
6
7 struct SparseTable{
8
9     int n, p;
10    vector<vector<int>> st;
11
12    SparseTable(vector<int> &arr){
13        n = (int) arr.size();
14        p = __lg(n);
15        st.assign(p+1, vector<int>(n, 0));
16
17        for(int j=0; j<n; j++) st[0][j] = arr[j];
18        for(int i = 1; i <= p; i++){
19            int k = (1ll << (i-1));
20            int lim = n - 2*k + 1;
21            for(int j=0; j<lim; j++){
22                st[i][j] = op(st[i-1][j], st[i-1][j+k
23            ]);
24        }
25    }
26
27    int query(int l, int r){
28        int len = r - l + 1;
29        int p_local = __lg(len), k = (1ll << p_local)
30        ;
31        return op(st[p_local][l], st[p_local][r - k +
32        1]);
33    };
34 };

```

7.7 Bit

```

1 // Time complexity: query and update  $\log N$ 
2 // 1-based indexing, [l, r]
3
4 struct BIT{
5
6     int n; vector<int> bit;
7     BIT(int n){
8         this->n = n;
9         bit.assign(n+1, 0);
10    }
11
12    void update(int i, int x){
13        while(i <= n){
14            bit[i] += x;
15            i += i & -i;
16        }
17    }
18
19    int query(int i){
20        int sum = 0;
21        while(i > 0){
22            sum += bit[i];
23            i -= i & -i;
24        }
25        return sum;
26    }
27
28    int queryRange(int l, int r){
29        return query(r) - query(l-1);
30    }
31
32    // retorna o k-ésimo elemento da bit
33    int find_kth(int k){
34

```

```

35     int idx = 0;
36     int max_bit = __lg(n);
37
38     for(int i = max_bit; i >= 0; i--){
39         int next_idx = idx + (1ll << i);
40         if(next_idx <= n and bit[next_idx] < k){
41             idx = next_idx;
42             k -= bit[next_idx];
43         }
44     }
45
46     return idx+1;
47 }
48
49 };

```

7.8 Monotonicstack

```

1 // Monotonic Stack, next greater number in a circular
  array
2 // Time Complexity: O(N)
3 // Space Complexity: O(N)
4 // Use cases: ring problems, finding next larger
  obstacle in a loop

```

```

5
6 for (int i = 0; i < 2 * n; i++) {
7     int num = a[i % n];
8     while (!st.empty() and a[st.top()] < num) { //
9         remove os menores
10         res[st.top()] = num; st.pop();
11     }
12     if (i < n) st.push(i);
13 }
14
15 // Monotonic Stack, next smaller element in a linear
  array
16 // Use cases: largest rectangle in histogram,
  subarray contribution
17
18 for (int i = 0; i < n; i++) {
19     while (!st.empty() and a[st.top()] > a[i]) { //
20         remove os maiores
21         res[st.top()] = a[i];
22         st.pop();
23     }
24     st.push(i);
25 }

```